Qwen

Achieving Kapwing Parity for Serbian Podcasts: A Tiered Open-Source Strategy Combining Multilingual LLMs, Local Serbian Models, and Non-LLM Algorithms

This report provides a comprehensive deep-dive analysis into viable open-source post-correction models and non-LLM techniques for a Serbian podcast clipping pipeline operating under strict CPU-only and no-paid-API constraints. The research directly supports architectural decisions for the `suggest.py` module and fix-queue latency management, ensuring post-correction remains functional within the project's defined operational envelope. The analysis evaluates a dual-track strategy: a primary path utilizing larger, general-purpose multilingual language models (LLMs) for maximum accuracy, and fallback paths including smaller, Serbian-specialized models for speed optimization and a robust suite of non-LLM methods to ensure complete runtime feasibility without any LLM dependency. The findings are grounded exclusively in the provided context blocks, synthesizing evidence from technical papers, model cards, and industry analyses to deliver actionable insights.

Tiered LLM Strategy for High-Accuracy Post-Correction

The investigation into open-source post-correction models reveals a clear strategic bifurcation, centered on a trade-off between the raw accuracy offered by large, general-purpose multilingual models and the speed advantages of more specialized, smaller architectures. Given the user's directive to prioritize accuracy first, this section focuses on identifying and evaluating the leading candidates in the larger model category. The core insight is that achieving significant error reduction through Large Language Models (LLMs) is not a matter of selecting any random large model; it requires a deliberate choice of a well-supported, multilingual foundation model combined with a targeted fine-tuning regimen specifically designed for ASR error correction. Naive application of off-the-shelf LLMs can lead to performance degradation and increased errors due to a lack of sensitivity to the specific nature of ASR mistakes [47].

The most promising candidates for the primary high-accuracy path include Google's Gemma 3 series, Alibaba's Qwen 3 series, Meta's Llama 3.1 model, and the European-focused TildeOpen model. These models stand out due to their extensive pre-training on multilingual data, broad language support claims, and availability under permissive licenses suitable for local deployment. Google's Gemma 3 family, available in sizes from 1B to 27B parameters, explicitly supports over 140 languages and has demonstrated strong performance on various benchmarks [30] [51]. Its multimodal variants can handle text and image input, but the text-only versions are relevant for transcription tasks [14]. Similarly, Alibaba's Qwen 3 series offers models ranging from 0.6B to 32B parameters, supporting 119 languages and trained on 36 trillion tokens, making it another top contender [14] [30]. Meta's Llama 3.1 8B Instruct model, while having licensing restrictions for very large-scale commercial use, is widely available and shows strong multilingual capabilities, making it a practical choice for many applications [29]. Finally, the TildeOpen-30B model emerges as a uniquely compelling option, as it was specifically designed to address the historical English-language bias in LLM training data and achieve state-of-the-art performance on European languages [15]. Its reported 41% greater efficiency for morphologically rich languages compared to competitors like Lama-3 suggests it may possess inherent advantages for processing the complex morphology of Serbian [15].

However, simply selecting one of these powerful models is insufficient. A critical study on Error Correction Language Models (ECLMs) highlights a crucial pitfall: using generic, instruction-tuned LLMs off-the-shelf for post-correction often leads to a significant increase in Word Error Rate (WER). In one experiment, the baseline ASR system had a WER of 5.3%, which escalated to 13.4% when corrected by a fine-tuned Llama-3.1-8B-Instruct model used in a zero-shot prompting setup [47]. This phenomenon occurs because general-purpose LLMs are not trained to understand the specific error patterns of ASR systems, such as phonetic confusions and homophones, and instead tend to over-correct based on their own world knowledge, introducing new errors [39] [47]. For instance, an LLM might correctly change 'baised' to 'biased' but incorrectly change 'raised' to 'baised' if presented with a garbled hypothesis, failing to preserve the acoustic evidence present in the original audio signal [47]. This underscores the necessity of specialization.

The most effective approach, therefore, is to move beyond naive prompting and embrace parameter-efficient fine-tuning (PEFT) or full fine-tuning on a dataset of synthetic or real ASR errors. Research demonstrates that this is where the true value of LLMs for post-correction lies. One study successfully integrated a decoder-only language model with a Whisper-based ASR system, achieving a CER/WER of 16.63% after fine-tuning on a multilingual dataset [49]. Another innovative framework, the Judge-Editor Agent (JEA),

uses an LLM to first judge the confidence of an ASR hypothesis and then selectively rewrite only the uncertain segments. When fine-tuned, this agent achieved a 14.51% relative WER reduction on a challenging dysarthric speech benchmark, proving the efficacy of a targeted, conditional correction strategy [52]. Furthermore, the LexBERT system, designed for correcting OCR inaccuracies in Serbian legal texts, provides a direct precedent for the proposed pipeline. It utilizes a RoBERTa-base model (SrBERTa v2) fine-tuned on Serbian legal corpora to generate top-20 correction suggestions for low-confidence words, demonstrating that domain-specific tuning is highly effective [44]. This approach aligns perfectly with the project's goal, suggesting a hybrid architecture where a powerful base LLM is augmented with context-aware suggestions generated by a smaller, Serbian-specific model.

Another viable strategy is to leverage sequence-to-sequence architectures specifically adapted for ASR post-editing, such as BART or T5. These models can be trained to map corrupted ASR hypotheses directly to clean transcripts [48]. A key insight from this line of research is that the quality of the training data is paramount. One study found that generating synthetic training data with a noisier Text-to-Speech (TTS) system, which produces a wider variety of ASR-like errors, resulted in a more effective ECLM than using a high-quality TTS system [47]. This implies that the project could create a powerful correction model by developing a simple script to introduce common Serbian ASR errors (homophones, phonetic substitutions) into a large corpus of clean Serbian text to generate its own training dataset. Augmenting these models with additional inputs, such as a grapheme-to-phoneme string, has also been shown to further reduce WER by providing explicit phonetic information to guide corrections [48].

To manage the high computational cost of running these large models on a CPU-only system, aggressive quantization is essential. Quantization reduces the precision of the model's weights (e.g., from 16-bit floating point to 8-bit or 4-bit integers), dramatically decreasing memory footprint and increasing inference speed at the cost of some accuracy [36]. Benchmarks conducted with the `llama.cpp` inference engine show that 4-bit GGUF quantization can yield a 4.3x improvement in throughput on a CPU compared to a FP16 baseline, reducing RAM usage significantly [25]. For example, a 7B model that would otherwise require 16GB of RAM can be run in approximately 4-5 GB with 4-bit quantization [25] [28]. While aggressive quantization can lead to accuracy loss, studies have shown that for certain structured tasks like intent extraction, even 3-bit quantization caused only a minor drop in accuracy, suggesting that for the task of post-correction, a 4-bit quantized model strikes an excellent balance between performance and resource consumption [25]. Frameworks like Intel's Extension for PyTorch (IPEX) offer advanced quantization methods like Weight-Only Quantization (WOQ) and SmoothQuant, which

are designed to maintain accuracy during the compression process, further enhancing the viability of CPU-based LLM inference [36].

In summary, the path to high-accuracy post-correction within the project's constraints involves a multi-step process. First, select a large, multilingual base model with strong language support, such as Gemma 3 4B, Qwen3 7B, or TildeOpen-30B. Second, develop a custom dataset of Serbian ASR errors, either synthetically or by collecting real-world examples. Third, fine-tune the selected model on this dataset using PEFT techniques like LoRA to make the process computationally tractable. Fourth, integrate the resulting model into a tiered correction pipeline, potentially using it in conjunction with a smaller Serbian-specific model for generating initial suggestions, as seen in the LexBERT architecture [44]. Finally, deploy the fine-tuned model using a CPU-optimized inference engine like `llama.cpp` with 4-bit GGUF quantization to ensure acceptable performance on the target hardware. This systematic approach moves beyond the ineffective practice of zero-shot prompting and leverages the full power of modern LLMs for the specific task of improving transcription quality.

Model Family	Representative Size(s)	Key Features & Rationale for Serbian Post-Correction
Gemma 3	4B, 12B, 27B	Broad multilingual support (>140 languages), strong benchmark performance, Apache 2.0 license. Fine-tuning has shown success in reducing CER/WER when paired with ASR systems [49] [51].
Qwen 3	Up to 32B	Extensive language support (119 dialects), trained on 36T tokens, Apache 2.0 license. Strong performance in multilingual contexts and efficient variants exist [14] [30].
Llama 3.1	8B	Strong multilingual capabilities, large community and tooling ecosystem. Fine-tuning has proven effective for selective rewriting in agentic frameworks [29] [52]. License has usage limits.
TildeOpen	30B	Specifically designed for European languages, addressing underrepresentation in LLM training. Shows superior efficiency for morphologically rich languages like Serbian [15].

Serbian-Specific Models as Speed-Oriented Fallbacks

While the primary objective is to achieve the highest possible accuracy through large multilingual models, maintaining a responsive user experience within the CPU-constrained environment necessitates a carefully architected fallback strategy. The second tier of the proposed solution involves leveraging smaller, Serbian-specific language models. These models, typically in the 3B to 8B parameter range, offer a compelling trade-off: they provide more contextual understanding and morphological awareness for the Serbian language than generalist models, while being significantly faster and less resource-intensive, making them ideal for near-real-time feedback in the user interface's

"fix-queue" [27] [28]. Their primary limitation is a narrower scope of knowledge and reasoning abilities compared to their larger counterparts, but for the task of targeted text correction, their strengths are well-suited [43].

The most prominent and relevant models identified for the Serbian language are BERTić and SRBerta. BERTić is a transformer-based language model with 110 million parameters, pre-trained on over 8 billion tokens of web-crawled text from Croatian, Bosnian, Montenegrin, and Serbian sources [17] [46]. Its architecture is based on the ELECTRA approach, which is known for its efficiency and strong performance on downstream NLP tasks. Crucially, BERTić has demonstrated state-of-the-art performance on several Serbian NLP benchmarks, including morphosyntactic tagging (microF1 score of 96.31%) and named entity recognition on social media text (microF1 score of 87.92%), showcasing its deep understanding of Serbian grammar and informal language use [46]. The existence of a Hugging Face repository for easy access makes it a practical choice for integration [46].

SRBerta is another powerful, domain-specific model developed for the Serbian language [42]. Based on the RoBERTa architecture, it was initially trained on the OSCAR corpus before being fine-tuned on a substantial dataset of 10,266 Serbian legal texts. This fine-tuning imbued the model with a strong ability to understand formal, complex Serbian and correct orthographic and morphological errors, a skill directly transferable to post-correction tasks [42]. The effectiveness of this approach was further validated by the LexBERT system, which employed SrBERTa v2 to generate context-aware correction suggestions for OCR'd Cyrillic text, achieving high accuracy by leveraging the model's deep linguistic knowledge [44]. The fact that this model was trained on legal documents, which feature precise and formal language, suggests it possesses a strong grasp of grammatical correctness that is highly valuable for cleaning up ASR transcripts [42].

Beyond these two major models, the research landscape for Serbian NLP identifies several other valuable resources that can form the basis of a robust correction pipeline. These include monolingual Serbian corpora like SrpKor2013 (~122 million words) and SrpLemKor (3.7 million lemmatized words), which are invaluable for training auxiliary models or validating outputs [17]. Furthermore, a collection of open-source Serbian morphological dictionaries and rule-based tools exists, such as SrbMD, srMWELex, and srLex, which can be used to build a dictionary-driven correction layer [17]. Tools for fundamental NLP tasks like tokenization (CLASSLA), POS tagging (HunPOS), and lemmatization (BTagger, 'lemma_spacylookup') are also available, providing the building blocks for a sophisticated non-LLM and LLM-hybrid system [17] [18].

The performance trade-offs of using these smaller models are well-documented. A 7-8B parameter model is generally considered to offer ~80% of the quality of much larger models for complex tasks but requires a correspondingly smaller hardware footprint, needing around 8-16GB of VRAM or 16GB of RAM when quantized [27]. A 3B model, like BERTić, can run comfortably on a modern laptop CPU with only 6-8GB of RAM, offering ~70% of the capability of a 7B model for tasks like summarization and email drafting [27]. On a CPU, a 3B model can achieve inference speeds of 50-200+ tokens/second, whereas a 7B model might operate at 20-80 tokens/second [28]. This speed difference is critical for the interactive "fix-queue" feature, where users expect immediate feedback. A 3B model can provide a suggestion almost instantaneously, while a 7B model might introduce a noticeable delay, degrading the user experience unless managed properly.

The optimal architectural placement for these models is as a dedicated "correction" component within a broader pipeline. Instead of being a universal replacement for the larger LLM, the smaller model acts as a fast, contextually aware specialist. The workflow would be as follows: the main ASR output is processed, and low-confidence words are flagged. These flagged words are then passed to the small Serbian-specific model. The model analyzes the surrounding context and generates a set of likely corrections. These suggestions are then presented to the user alongside those from the larger, more powerful multilingual LLM. This hybrid approach combines the best of both worlds: the speed and linguistic specificity of the smaller model for quick, reliable corrections, and the vast knowledge base and creative problem-solving of the larger model for more difficult cases. The LexBERT system serves as a perfect blueprint for this synergy, using a fine-tuned RoBERTa model to generate suggestions that are then evaluated by a distance metric [44]. This layered approach ensures that the system remains both accurate and responsive, fully respecting the project's CPU-only constraint.

Model Name	Type / Base Architecture	Parameters	Key Strengths for Serbian Post-Correction	Availability
BERTić	Transformer (ELECTRA)	110M [46]	Pre-trained on 8B+ tokens of South Slavic text, excels at morphosyntactic tagging and NER. Proven on Serbian benchmarks. [46]	Hugging Face
SRBerta / SrBERTa v2	Transformer (RoBERTa)	Base model details available [42] [44]	Fine-tuned on Serbian legal texts, strong at correcting OCR/ morphological errors. Used effectively in the LexBERT system. [42] [44]	Not specified
TildeOpen-30B	Transformer	30B	Designed for European languages, shows high efficiency for morphologically rich languages. Broader multilingual support than pure Serbian models. [15]	Hugging Face
SmolLM3-3B	Transformer	3.1B [13]	Fully open, transparent training pipeline. Outperforms other models of similar size on some reasoning tasks. Good candidate for a fast, lightweight correction model. [13] [29]	Hugging Face

A Suite of Non-LLM Correction Techniques

Before resorting to computationally expensive Large Language Models, a powerful and essential layer of correction can be implemented using a suite of non-LLM techniques. This approach is critical for fulfilling the project's "open-source only" and "CPU-only" contract, as it provides a mechanism for error correction that operates entirely without any LLM dependency. These methods excel at handling common types of errors produced by ASR systems, such as spelling mistakes, morphological variations, and phonetic confusions. By integrating a hybrid system that first applies these fast, deterministic checks and only then escalates to probabilistic LLM-based suggestions, the overall efficiency and reliability of the post-correction pipeline can be significantly enhanced. This strategy directly addresses the need for a fallback mechanism that is always available, regardless of the availability or performance of LLMs [20].

One of the most effective non-LLM techniques for this task is fuzzy string matching. These algorithms assess the "distance" between two strings, making them ideal for detecting and correcting typos and minor spelling variations [20]. The Levenshtein Distance algorithm, which calculates the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one word into another, is a foundational method in this category [20]. For example, if the ASR outputs "przeo sam životar" and the intended word is "prošao sam života," a fuzzy matcher can identify "przeo" as being one substitution away from "prošao." Libraries like `fuzzywuzzy` or `python-Levenshtein` can efficiently apply this logic across a dictionary of valid words to find the closest match. This method is particularly effective for catching simple character-level errors that are common in ASR output [20].

Complementing fuzzy matching is phonetic matching, which is crucial for correcting errors based on pronunciation rather than spelling. This is especially relevant for homophones (words that sound the same but are spelled differently) and garbled output where the exact spelling is unclear. Several phonetic algorithms are well-suited for this purpose. Metaphone 3, for instance, is specifically noted for its ability to handle personal names and surnames with high precision and is adjusted for various language families, including European and Slavic ones [21]. The Beider-Morse Phonetic Matching System is another powerful tool, designed to solve irrelevant matches by applying extensive language-specific pronunciation rules, which is highly applicable to proper nouns and names in podcasts [21]. Daitch-Mokotoff Soundex is also mentioned as a phonetic algorithm suitable for Slavic languages [21]. These algorithms work by converting words into a phonetic code. A misspelled word and its correct form will often share a similar phonetic code, allowing the system to identify them as potential matches even if their

spellings differ significantly. This technique is frequently combined with fuzzy matching to create a robust system capable of handling both spelling and pronunciation errors [19] [37].

For more context-aware corrections, particularly those involving morphology, statistical language models and morphological tools are indispensable. An n-gram language model, built on a large Serbian corpus like SrpKor2013, can provide a fast and effective way to penalize invalid or unlikely word sequences generated by the ASR [17] [26]. If the ASR produces a sequence of words that has a very low probability according to the n-gram model (e.g., "the are oul criminals"), the model can flag it as suspicious. The KenLM library is a popular and efficient tool for building and querying such models, which can function as a lookup table for word probabilities [26]. This method is computationally inexpensive and highly effective at catching non-word errors and basic grammatical mistakes that violate syntactic norms [26].

Further enhancing the system's linguistic understanding involves leveraging morphological analyzers like stemmers and lemmatizers. Stemming reduces words to their root form by removing suffixes (e.g., "running" becomes "run"), while lemmatization reduces them to their base or dictionary form (e.g., "ran" becomes "run") [16]. Open-source tools like jLemmaGen and Snowball have built-in support for the Serbian language [16]. By normalizing words to their base forms, the system can compare them against a canonical dictionary of valid lemmas, catching inflectional errors where the ASR might have misidentified a verb tense or noun case. Rule-based lemmatizers, such as the 'lemma_spacylookup' model available in the Spark NLP library, offer production-ready solutions for this task [18]. These tools can be integrated into a pipeline to validate the morphological correctness of words flagged by the confidence scores.

Finally, a comprehensive non-LLM correction suite should be designed as a hybrid, cascading system. The recommended workflow would be: 1. **Initial Filtering:** Use the ASR's native confidence scores to identify low-probability words. 2. **Fuzzy Matching:** Apply a Levenshtein-distance-based checker against a standard Serbian dictionary to find likely typos. 3. **Phonetic Matching:** If fuzzy matching fails, pass the low-confidence word through a phonetic algorithm (e.g., Metaphone 3) to find homophonic or phonetically similar candidates. 4. **Statistical Validation:** Use an n-gram model to check if the corrected word fits grammatically within its surrounding context. 5. **Morphological Validation:** Use a lemmatizer/stemmer to verify the corrected word's morphological validity. This layered approach mirrors the logic used in advanced search and retrieval systems and allows for a flexible and robust correction process that leverages the strengths of multiple non-LLM techniques without requiring the heavy computational

overhead of an LLM [20]. This entire system can be implemented with Python libraries such as `cyrtranslit` for transliteration, `hunspell` for dictionary-based checking, and `python-levenshtein` or `fuzzywuzzy` for similarity matching, ensuring all components are open-source and CPU-friendly.

Technique	Algorithm/Tool Example	Primary Function	Applicability to Serbian Transcription
Fuzzy String Matching	Levenshtein Distance	Detects typos and minor spelling variations by calculating edit distance.	Corrects simple character-level errors common in ASR output (e.g., 'životar' vs. 'života'). [20]
Phonetic Matching	Metaphone 3, Beider-Morse	Identifies words that sound alike, useful for homophones and garbled pronunciation.	Excellent for resolving homophones and transcriptions of names where spelling is ambiguous. [21] [37]
Statistical Language Modeling	n-gram (with KenLM)	Penalizes unlikely or ungrammatical word sequences based on a large corpus.	Catches non-word errors and basic syntactic mistakes by checking local context. [17] [26]
Morphological Analysis	Snowball, jLemmaGen, HunPOS	Reduces words to their root/base form (stemming/lemmatization) for validation.	Validates morphological correctness (cases, tenses, genders) against a canonical dictionary. [16] [17]
Rule-Based Dictionary	Hunspell-sr	Provides a dictionary of correctly spelled words for real-time checking.	Fast and effective for catching standard spelling errors against a known vocabulary.

Performance Analysis and Runtime Feasibility on CPU Hardware

Executing a sophisticated post-correction pipeline on a CPU-only machine imposes stringent constraints on performance, making a detailed analysis of model characteristics, quantization effects, and hardware dependencies absolutely critical. The success of the entire project hinges on whether the chosen models can be run in a timely manner to keep pace with the rest of the video processing workflow. This section delves into the quantitative aspects of LLM inference on CPUs, examining metrics like tokens per second, RAM requirements, and the profound impact of quantization, thereby grounding the architectural choices in tangible performance realities.

The performance of any LLM on a CPU is primarily dictated by its parameter count. As a general rule, larger models are slower and consume more memory. On a typical consumer-grade CPU, a 1-billion-parameter model can achieve 50-200+ tokens per second, while a 7-8 billion parameter model operates at a more modest 20-80 tokens per second [28]. For the proposed pipeline, this translates directly to the responsiveness of the "fix-queue" UI. A 3B model could suggest a correction for a sentence in under a second,

providing a fluid user experience. A 7B or 8B model might take several seconds, which, while acceptable for a batch process, could feel sluggish in an interactive context. Therefore, the selection of a model size represents a direct trade-off between potential accuracy and perceived latency. For the CPU-constrained environment, models in the 3B-7B range represent a practical sweet spot, offering a good balance of capability and speed [27] [28].

Memory (RAM) is another critical bottleneck. An unquantized 7B model requires approximately 8-16 GB of VRAM or 16 GB of RAM, while a 13B model needs double that [27] [28]. Since the project mandates a CPU-only environment, system RAM becomes the limiting factor. Even with quantization, memory usage remains a concern. A 4-bit quantized model requires about 0.5-1 GB of storage per billion parameters [28]. Thus, a 7B model quantized to 4-bits would occupy roughly 3.5-7 GB of RAM, plus overhead for the operating system and other processes. This leaves limited headroom for other parts of the pipeline, such as the ASR engine itself and the final video rendering stages. This reality reinforces the wisdom of the tiered approach: using a smaller, highly optimized 3B model for the interactive fix-queue, while reserving larger 7B+ models for background, asynchronous correction jobs that do not block user interaction [35].

Quantization is not merely an optimization but a prerequisite for feasible CPU-based LLM inference. It is the process of reducing the numerical precision of a model's weights, which dramatically lowers computational complexity and memory bandwidth requirements [36]. The most common quantization formats are INT8 (8-bit integers) and INT4 (4-bit integers). While converting from FP32 to INT8 offers a theoretical 4x reduction in computation, actual latency improvements are less dramatic due to other factors [36]. However, benchmarks show that moving from FP16 to 4-bit GGUF quantization on a CPU using `llama.cpp` can yield a 4.3x improvement in throughput [25]. This is a crucial finding, as it demonstrates that modern quantization techniques can make models orders of magnitude faster on CPUs. The choice of quantization variant is important; for example, the GGUF `Q4_K_M` format is often cited as providing the best balance of high throughput, low latency, and minimal accuracy loss for general-purpose tasks [25].

The hardware platform itself plays a significant role. While the pipeline is CPU-only, different processors offer varying levels of acceleration. Modern Intel Xeon CPUs with Advanced Matrix Extensions (AMX) can see up to a 10x reduction in latency for encoder-dominant tasks, which LLM inference resembles, when using optimized software like Intel Extension for PyTorch (IPEX) [24]. On a typical AMD EPYC processor, a 4-bit GGUF quantized 7B model achieved 19.5 tokens/second, a 4.3x improvement over its FP16

counterpart [25]. For developers working on heterogeneous server environments, extensive benchmarks have been conducted using `llama.cpp` across over 2000 cloud servers, confirming that even small models can run on a wide array of cheap CPU instances, highlighting the cost-efficiency of CPU-based inference at scale [23]. For local development, a modern multi-core CPU is sufficient, but performance will be heavily dependent on correctly configuring the number of threads used for inference, often via environment variables like `OMP_NUM_THREADS` [9].

Given these performance realities, the system architecture must be designed to mitigate latency. A synchronous request-response model, where the user clicks a button and waits for an LLM to generate a suggestion, is unacceptable. The adversarial review adaptation correctly identifies the need for an asynchronous, batched approach. After the initial transcription completes, all low-confidence words can be collected and sent to the LLM in a single, batched prompt. This background job runs concurrently with other pipeline steps, and the suggestions are made available to the UI as soon as they are ready. This design pattern hides the inherent latency of LLM inference behind a fast, non-blocking interface, ensuring a smooth user experience [35]. The `faster-whisper` library, which uses the CTranslate2 inference engine, already employs a batched inference pipeline that can significantly improve throughput on a CPU, a principle that should be extended to the post-correction stage [9]. By combining a well-chosen, quantized model with an intelligent, asynchronous architecture, the project can deliver the benefits of LLM-powered correction without violating its strict CPU-only runtime contract.

Metric	Small Model (e.g., 3B)	Medium Model (e.g., 7B)	Large Model (e.g., 14B+)
Tokens/Second (CPU)	50 - 200+ [28]	20 - 80 [28]	< 20 (highly variable)
RAM (Unquantized)	~4 - 8 GB [28]	~8 - 16 GB [28]	> 16 GB [28]
RAM (4-bit Quantized)	~1.5 - 4 GB [25] [28]	~3.5 - 8 GB [25] [28]	~7 - 16 GB [25] [28]
Suitable Use Case	Interactive fix-queue, real-time suggestions.	Batch correction, complex context analysis.	Offline analysis, document-level rewriting.
Key Optimization	Maximize throughput for interactivity.	Balance accuracy and speed for batch jobs.	Focus on accuracy, accept long processing times.

Landscape of Alternative Serbian Speech Technologies

The project's requirement to validate runtime feasibility extends beyond post-correction to encompass the entire speech technology stack, including Automatic Speech Recognition (ASR) and forced alignment models. The plan currently relies on `sr-v3-ct2`, a fine-tune of Whisper-large-v3, and `classla/wav2vec2-xls-r-juznevesti-sr` for alignment. While these are strong, established models, the rapid evolution of speech AI in 2025-2026 warrants a broader landscape analysis to ensure the chosen stack remains state-of-the-art and to identify potential future upgrades. The evaluation must focus on models that are open-source, available for local deployment, and demonstrate strong performance on the specific challenge of Serbian audio, particularly in the noisy, conversational domain of podcasts.

The primary ASR model identified for the pipeline, `sr-v3-ct2`, is a CT2-converted version of a Sagicc-class fine-tune of `whisper-large-v3-sr` [1]. Its strength lies in its heritage from the powerful Whisper-large-v3 architecture, which is known for its versatility and high accuracy across many languages [4]. The fine-tuning on datasets like CV13+FLEURS+Južne vesti gives it a tailored advantage for Serbian [1]. However, its self-reported WER of approximately 4% is likely based on clean, read-speech corpora, a condition that does not reflect the acoustic reality of podcast audio, which often contains overlapping speakers, background noise, and varied microphone quality [1] [7]. A rigorous bake-off on representative `.ts` files, as planned in Wave 0, is essential to establish a realistic baseline WER, which is expected to be significantly higher [1] [7].

For alternative ASR models, the 2025-2026 landscape introduces several notable contenders. NVIDIA's Canary-Qwen 2.5B model has emerged as a leader on public ASR leaderboards, achieving a 5.63% average WER with a highly efficient architecture [1]. It combines a FastConformer encoder with a Qwen language model decoder and can be up to 10x faster than similarly accurate models, though it requires NVIDIA's NeMo framework and GPU resources for optimal performance [1]. IBM's Granite Speech models, including a 1B and 8B parameter version, also offer strong performance with an Apache 2.0 license, making them suitable for local deployment [1]. Alibaba's Qwen3-ASR is another significant player, offering both accuracy-optimized (1.7B) and efficiency-optimized (0.6B) variants [1]. However, its documented strengths lie in challenging acoustic scenarios like elderly/child speech and Asian languages, and none of the listed models support Serbian [3]. The emergence of models like Moonshine, which is purpose-built for edge devices like Raspberry Pis, signals a trend towards highly efficient, low-footprint ASR, but its primary focus on English means it is not a direct candidate for this

project [4]. The most promising path forward appears to be continuing with the Whisper-family models, given their strong multilingual capabilities and the availability of open-weight, locally-deployable versions like `faster-whisper` [4]. The plan to benchmark `whisper-large-v3-turbo CT2` as a speed alternative is a sound strategy to balance performance and throughput [1].

For the crucial word alignment step, the plan's reliance on `classla/wav2vec2-xls-r-juznevesti-sr` is well-founded. This model is a dedicated Serbian ASR model fine-tuned on Južne vesti news audio, providing a Latin alphabet vocabulary suitable for alignment tasks [5]. The Wav2Vec2 architecture, which uses Connectionist Temporal Classification (CTC), is inherently suited for sequence-to-sequence problems like alignment [4][5]. The plan's fallback chain, which includes smaller wav2vec2 models and eventually the `stable-ts` library, is a robust strategy to handle potential failures on noisy audio [1]. The `stable-ts` aligner, which uses Dynamic Time Warping (DTW), is a particularly valuable fallback as it requires no additional pre-trained model, making it a stable and reliable component of the pipeline [1].

Looking ahead, the broader trends in speech technology reinforce the project's architectural choices. The continued dominance of the Wav2Vec 2.0 framework, evidenced by millions of downloads on Hugging Face, confirms its suitability for fine-tuning on low-resource languages like Serbian [7]. Its ability to achieve high accuracy with as little as 10 minutes of labeled data is a key advantage [7]. The development of massive multilingual models like Meta's Omnilingual ASR, which aims for coverage of over 1,600 languages, points toward a future where highly capable, zero-shot models may become more accessible [2]. However, for the immediate term, specialized, fine-tuned models remain the most reliable path to high-quality results. The project's plan to periodically re-evaluate the ASR and alignment landscape is prudent, but the current choices of `sr-v3-ct2` and `wav2vec2-xls-r-juznevesti-sr` are well-positioned and backed by strong evidence of their effectiveness for the Serbian language [1][5].

Model / Technology	Type	Key Feature for Serbian Pipeline	Potential Limitation
sr-v3-ct2	ASR (Whisper-based)	Fine-tuned on Serbian data, high-quality Cyrillic output.	Performance on noisy podcast audio is unknown; self-reported WER is for clean speech.
wav2vec2-xls-r-juznevesti-sr	Alignment (Wav2Vec2-based)	Dedicated Serbian model with Latin vocab, plug-and-play with whisperx.	May struggle with low SNR audio; requires a fallback plan.
Canary-Qwen 2.5B	ASR (Multilingual)	State-of-the-art leaderboard performance, high speed.	Primarily designed for NVIDIA hardware; lacks confirmed Serbian performance.
Granite Speech 1B/8B	ASR (Multilingual)	Strong performance, Apache 2.0 license, smaller variant for constrained environments.	General multilingual model; may not be optimized for Serbian.
Qwen3-ASR 0.6B/ 1.7B	ASR (Multilingual)	Efficiency-optimized variants available for high-throughput tasks.	Optimized for Asian languages and specific domains, not Serbian.
Omnilingual ASR	ASR (Multilingual)	Massive language coverage (1,600+), zero-shot generalization.	Very large models may be too slow for CPU-only inference; performance on low-resource languages can vary.

Synthesis and Architectural Recommendations

This research has systematically evaluated open-source post-correction models and non-LLM techniques for a CPU-constrained Serbian podcast clipping pipeline. The analysis confirms that achieving high-quality, automated post-correction without paid APIs is not only feasible but can be accomplished through a sophisticated, tiered architectural strategy that balances accuracy, speed, and resource constraints. The central recommendation is to implement a multi-layered correction system that intelligently combines the raw power of fine-tuned multilingual LLMs, the contextual intelligence of smaller Serbian-specific models, and the speed and reliability of a robust non-LLM toolkit. This approach directly addresses the project's core research goal and provides a resilient, scalable, and fully open-source solution.

First, the primary path for high-accuracy post-correction should leverage a large, general-purpose multilingual LLM, such as Google's Gemma 3 4B or Alibaba's Qwen3 7B. The evidence overwhelmingly indicates that naive, zero-shot prompting of these models is ineffective and can degrade transcription quality [47]. Therefore, the critical next step is to acquire a suitable base model and create a custom training dataset of Serbian ASR errors. This dataset should be generated synthetically by introducing common Serbian errors (homophones, phonetic confusions) into a large corpus of clean Serbian text [47]. The model should then be fine-tuned on this data, preferably using a parameter-efficient method like LoRA, to specialize it for the task of ASR post-correction. To manage

computational costs on a CPU, this fine-tuned model must be aggressively quantized to at least 4-bits using a format like GGUF Q4_K_M, which has been shown to offer a significant throughput boost with minimal accuracy loss [25].

Second, a fallback path for speed-optimized correction should be implemented using smaller, Serbian-specific models. BERTić and SRBerta are the most promising candidates, offering deep morphological understanding of the Serbian language that generalist models may lack [42] [46]. These models, being smaller (110M to ~768 hidden size), will execute much faster on a CPU, making them ideal for providing near-instantaneous suggestions in the interactive "fix-queue" portion of the UI [27] [28]. The architecture should be designed to use this model as a fast, context-aware specialist, generating initial correction candidates that can then be refined or supplemented by the larger, more powerful LLM if needed.

Third, a comprehensive non-LLM suite must form the foundational layer of the correction pipeline. This layer ensures that the system remains fully functional and provides value even if LLMs are unavailable or too slow. This suite should be a hybrid system combining several techniques: `cyrtranslit` for deterministic Cyrillic-to-Latin conversion [1]; Hunspell-sr for dictionary-based spelling checks; fuzzy string matching (Levenshtein Distance) to catch typos [20]; and phonetic algorithms like Metaphone 3 to resolve homophones [21]. These methods should be applied in a cascading fashion to low-confidence words flagged by the ASR, creating a fast and efficient first-pass correction engine that handles a majority of common errors without any LLM overhead.

Finally, the entire architecture must be designed with CPU limitations in mind. The use of `faster-whisper` (which relies on the CTranslate2 inference engine) for the initial transcription is a cornerstone of this strategy, as it can provide up to a four-fold speedup over the original implementation [9]. For the LLM-based tiers, an asynchronous, batched processing model is non-negotiable to prevent UI blocking. All low-confidence words should be collected and sent to the LLM as a single batched request after transcription completes, with the results made available to the UI as they arrive [35]. This design decouples the slowest part of the process from the user's immediate interaction, ensuring a responsive and usable experience. The project's initial "bake-off" phase, which empirically tests models on actual podcast audio to establish realistic WER baselines, is a crucial step that validates this performance-oriented approach [1].

By adopting this tiered, performance-conscious architecture, the Serbian Clip Studio pipeline can achieve a level of post-correction quality that rivals proprietary systems, all while adhering strictly to its open-source and CPU-only operational contract.

Reference

1. Best open-source speech-to-text models in 2026 https://www.gladia.io/blog/best-open-source-speech-to-text-models
2. Omnilingual ASR: open-source multilingual speech ... https://www.facebook.com/groups/3670562573177653/posts/4372837766283460/
3. Comparing Speech-to-text Models | CCV AI Services https://docs.ccv.brown.edu/ai-tools/services/transcribe/comparing-speech-to-text-models
4. Best Open Source Speech-to-Text Models in 2026 https://www.assemblyai.com/blog/top-open-source-stt-options-for-voice-applications
5. classla/wav2vec2-xls-r-juznevesti-sr https://huggingface.co/classla/wav2vec2-xls-r-juznevesti-sr
6. ASR training dataset for Serbian JuzneVesti-SR v1.0 https://www.clarin.si/repository/xmlui/handle/11356/1679
7. Wav2Vec 2.0 Statistics And User 2026 https://www.quantumrun.com/consulting/wav2vec-2-0-statistics/
8. CTranslate2 - Fast inference engine for Transformer models https://github.com/opennmt/ctranslate2
9. Faster Whisper transcription with CTranslate2 https://github.com/SYSTRAN/faster-whisper
10. Qwen2.5-LLM: Extending the boundary of LLMs https://qwenlm.github.io/blog/qwen2.5-llm/
11. Best Open Source LLMs of May 2026: We Reviewed 7 ... https://fireworks.ai/blog/best-open-source-llms-may-2026
12. State of the Art in Text Classification for South Slavic ... https://arxiv.org/html/2511.07989v2
13. The Best Open-Source Small Language Models (SLMs) in ... https://www.bentoml.com/blog/the-best-open-source-small-language-models
14. Best Open Source LLMs: Complete 2026 Guide https://contabo.com/blog/open-source-llms/
15. LT https://tilde.ai/lt/tildeopen-llm/
16. Language Analyzers & Tokenizers for ELK https://logz.io/blog/language-analyzers-tokenizers-not-built-elasticsearch-where-find-them/

17. A Survey of Resources and Methods for Natural Language ... https://arxiv.org/html/2304.05468v1
18. Lemmatizer (Serbian, SpacyLookup) | lemma_spacylookup https://sparknlp.org/2022/03/03/lemma_spacylookup_sr_3_0.html
19. Transliteration Matching across all Non-Latin Data Sets https://www.datactics.com/blog/cto-vision/transliteration-matching/
20. Fuzzy Matching and Semantic Search https://ipullrank.com/fuzzy-matching-semantic-search
21. Phonetic Matching Algorithms https://medium.com/@ievgenii.shulitskyi/phonetic-matching-algorithms-50165e684526
22. Benchmarking AI inference on CPUs https://next.redhat.com/2026/05/28/benchmarking-ai-inference-on-cpus-a-transparent-blueprint-for-the-enterprise/
23. LLM Inference Speed Benchmarks https://sparecores.com/article/llm-inference-speed
24. AI Inferencing on Intel CPU-Powered Lenovo Servers https://lenovopress.lenovo.com/lp2167-ai-inferencing-on-intel-cpu-powered-servers-accelerating-llm-performance
25. Performance Trade-Offs of Optimizing Small Language ... https://www.mdpi.com/2504-2289/10/5/155
26. Boosting Wav2Vec2 with n-grams in Transformers https://huggingface.co/blog/wav2vec2-with-ngram
27. LLM Parameters Explained: 1B to 1T Model Sizes https://iternal.ai/llm-parameter-size-guide
28. LLM Model Parameters 2026: 7B-70B https://local-ai-zone.github.io/guides/what-is-ai-model-3b-7b-30b-parameters-guide-2025.html
29. 15 Best Lightweight Language Models Worth Running in ... https://www.premai.io/blog/best-lightweight-language-models-worth-running/
30. agi-templar/Awesome-Small-Language-Model https://github.com/agi-templar/Awesome-Small-Language-Model
31. Serbian SuperGLUE: Towards an Evaluation Benchmark ... https://aclanthology.org/2026.loreslm-1.30.pdf
32. Why LLM Performance Drops in Non-English Languages https://lilt.com/blog/multilingual-llm-performance-gap-analysis
33. Multilingual Evaluations in LLMs — a comparison https://medium.com/@vbsowmya/multilingual-evaluations-in-llms-a-comparison-1d58b0fd9848
34. Multilingual AI: Challenges and Solutions for Serbian https://www.linkedin.com/posts/djordje-medakovic-11b51b21_building-ai-apps-in-serbian-activity-7370289892230139904-soaP

35. Understand LLM latency and throughput metrics https://docs.anyscale.com/llm/serving/benchmarking/metrics
36. Improving LLM Inference Latency on CPUs with Model ... https://towardsdatascience.com/improving-llm-inference-latency-on-cpus-with-model-quantization-28aefb495657/
37. Phonetic Algorithms to Optimize Search Engines, Spell ... https://medium.com/@priom7197/%EF%B8%8Fphonetic-algorithms-to-optimize-search-engines-spell-correction-suggestions-and-more-65e06dc991d0
38. How I Automated My Podcast Transcript Production With Local ... https://den.dev/blog/how-i-automated-podcast-transcription-with-local-ai/
39. Contextualized Spoken-to-Written Conversion of ASR ... https://arxiv.org/html/2408.09688v2
40. BTLM-3B-8K: 7B Parameter Performance in a ... https://arxiv.org/html/2309.11568
41. Best Small Language Models for Accuracy and Enterprise ... https://medium.com/@darrenoberst/best-small-language-models-for-accuracy-and-enterprise-use-cases-benchmark-results-cf71964759c8
42. SRBerta—A Transformer Language Model for Serbian ... https://www.mdpi.com/2078-2489/15/2/74
43. Getting High-Quality Output from 7B Models: A Production ... https://dev.to/superorange0707/getting-high-quality-output-from-7b-models-a-production-grade-prompting-playbook-21hi
44. Improving Text Recognition Accuracy for Serbian Legal ... https://www.mdpi.com/2076-3417/15/2/615
45. Which one Performs Better? Wav2Vec or Whisper? ... https://arxiv.org/html/2508.09957v1
46. BERTić - The Transformer Language Model for Bosnian, ... https://arxiv.org/html/2104.09243v1
47. Revisiting ASR Error Correction with Specialized Models https://arxiv.org/html/2405.15216v2
48. ASR Error Correction: Methods & Advances https://www.emergentmind.com/topics/asr-error-correction-aec
49. Qwen vs. Gemma Integration with Whisper https://arxiv.org/html/2506.13596v1
50. Fine-Tuning Gemma 3n for Speech Transcription https://debuggercafe.com/fine-tuning-gemma-3n-for-speech-transcription/
51. Gemma 3: Google's all new multimodal, multilingual, long ... https://huggingface.co/blog/gemma3
52. LLM-Agent Post-ASR Correction Beyond WER https://arxiv.org/html/2601.21347v1

53. Released the Highest-Accuracy Japanese ASR Model for ... https://neosophie.com/en/blog/20260427-qwen-finetuned-model